12 bodů
Ben

KSP-H 38-1-3 Mediánový poddaný

Nakonec hodnotím řešení v $\mathcal{O}$(K log Pmax), které si zmínil na konci druhé stránky. Jemu ale chybí analýza časové složitosti.

Řešení:

Níže je stručná struktura mého řešení (nejedená se o pseudokód, je to spíše strukturovaný slovní popis). Ještě je dobré zdůraznit, že ve výsledném kódu, který také vložím, počítám do určité míry s tím, že je archiv v kraji podle pracovitosti setříděn. Pokud ne, posel bude mít v prvních K průchodech napilno a u všech krajů archivy lokálně setřídí podle pracovitosti vzestupně. Proto dále v řešení už s tímto budu počítat.

Struktura algoritmu:

Udržuj si min a max pro nejnižší a nejvyšší pracovitost z mediánů v K[i];
Udržuj si smlr_c a lrgr_c pro celkový počet poddaných menších nebo větších odhad;
Dokud není medián nalezen:
    Udržuj si pole velikosti K obsahující objekt Range s pěti proměnnými (rozsahy);
    Udržuj si proměnnou bm_c pro pro součet poddaných mezi rozsahy;
    Udržuj si e_s a e_l obsahující nižší a vyšší pracovitost nejbližší k odhadu mediánu
    Procházej všechny kraje K:
        Při prvních K průchodech:
            Pokud je třeba setřídit archiv, setřiď jej tam;
            Ulož do pole rozsahy: medián, P[N/4], P[3*N/4], počty poddaných mezi;
            Aktualizuj min a max pokud je medián menší nebo vyšší;
            Sčítej počty poddaných mezi min a max pro celé K += K[i];
            Při posledním průchodu:
                Vypočti odhad mediánu pomocí **CDF** z hodnot rozsahů;
        Pro všechny ostatní průchody:
            Procházej jednotlivé poddané N:
                Pokud je P[i,j] < odhad mediánu: smlr_c++;
                Jinak pokud je P[i,j] > odhad mediánu: lrgr_c++;
                Jinak: poddaný s hodnotou odhadu je v kraji;
                Aktualizuj e_s a e_l podle odhadu mediánu;
                Získej nové rozsahy pro každý kraj mezi min a max;
                Sčítej počty poddaných mezi min a max pro celé K += K[i];
                Sčítej počty poddaných s P < min pro celé K
    Pokud byl výsledek odhadu mediánu získán (po 2K průchodech):
        Pokud smlr_c > lrgr_c:
            Vypočti kde je v rozsahu medián K*N/2 - počet poddaných s P < min;
            Vypočti nový odhad pomocí **CDF** z rozsahu a pozice mediánu v nich;
            Aktualizuj max na hodnotu poddaného nejblíže k odhadu e_s;
        Pokud smlr_c < lrgr_c:
            Vypočti kde je v rozsahu medián K*N/2 - počet poddaných s P < min;
            Vypočti nový odhad pomocí **CDF** z rozsahu a pozice mediánu v nich;
            Aktualizuj min na hodnotu poddaného nejblíže k odhadu e_l;
        Jinak: Medián nalezen!

Nemyslím si, že je vhodné kód strukturovat jako jednu velkou smyčku, když každá iterace dělá něco jiného. Navíc jeden krok binárního vyhledávání provádíš přes více iterací smyčky. Mnohem přirozenější by mi přišlo popis i implementaci algoritmu strukturovat zhruba takto:

min, max ← počáteční hodnota
dokud není nalezen medián:
spočítáme rozsahy
určíme odhad pomocí CDF
spočítáme menší a větší prvky
zmenšíme interval nebo zkončíme

7281312229010 Radek Zach 38-1-3 1/11

Osobně bych dal pseudokód až na konec a soustředil se na slovní popis, místo toho, abych napsal nejprve pseudokód a pak ho komentoval. Ale záleží určitě na osobní preferenci a na algoritmu, někdy je potřeba algoritmus vidět celý, aby ho bylo možné pochopit.

Toto je pouze stručný popis, který má ukázat jeho myšlenku a rozhodně není v tomto stavu funkční. Možná jste si všimli, že tato ukázka připomíná z části binární vyhledávání, nebo spíše vše okolo je tomu dosti podobné. A ano, je tomu tak, ovšem ta nejdůležitější část, která zároveň určuje to, jak rychle medián najdeme (výpočet odhadu mediánu), je zde zcela odlišná.

Tohle je dobré, algoritmy je vhodné rozložit na části. V popisu algoritmu je vhodné nejprve popsat na vysoké úrovni, co algoritmus dělá, a až poté rozebírat, jak jednotlivé části fungují. (Nebo nejprve popsat jednotlivé části a pak je spojit dohromady, pokud je možné jednotlivé části snadno popsat odděleně.) Takže začít s tím, že budeme provádět binární vyhledávání s jinou volbou porovnávaného prvku je dobrý nápad.

Reprezentace vyslání posla , nebo velikost zprávy zde není nějak jasně viditelná. Avšak právě každá iterace cyklu procházející jednotlivé kraje K[i], vyslání posla představuje. Velikost zprávy se liší pouze pro prvních K průchodů, ve všech ostatních zůstává stejná a jejich velikost splňuje podmínky zadání. Pro prvních K průchodů bude zpráva obsahovat 5 hodnot. Tento soubor pěti hodnot budeme nazývat souhrnně rozsahy. První tři hodnoty budou obsahovat pracovitosti poddaných v daném kraji K[i], kteří se nachází na konkrétních pozicích v setříděném archivu, respektive pro první hodnotu s názvem **mid_val** získáme pracovitost jež leží na pozici N/2-1 pokud je počet poddaných sudý nebo N/2 pokud je počet lichý (získáme tak pracovitost, která je mediánem kraje).

Teď přeskočíš k tomu, jak získáme hodnoty potřebné k spočítání odhadu. Není ale vůbec jasné, co tyto hodnoty reprezentují, nebo k čemu jsou. Čtenář si musí domyslet, že dělíš poddané na čtvrtiny a pamatuješ si velikost a hranice prostředních dvou.

Další dvě hodnoty budou obsahovat velikosti P na pozicích N/4 a 3*N/4 (hodnota **left_val** → medián poddaných pro něž platí P[i] < **mid_val**, hodnota **right_val** naopak bude mediánem z P[i] > **mid_val**). Pro jednoduchost je budeme nazývat obecně **m_mediáni**. Zbylé dvě hodnoty označíme jako **left_mid_count**, obsahující počet poddaných s velikostmi P[i] mezi **left_val** a **mid_val**, a nakonec **mid_right_count** s počtem poddaných kteří se nachází mezi mediánem celého kraje a horní čtvrtinou. Takto pro každý kraj při jejich prvním průchodu získáme hodnoty, které si budeme při návratu posla ke každému kraji ukládat (objekt obsahující těchto 5 hodnot nazvěme Range).

Průběžně si také mezi průchody budeme aktualizovat hodnoty **min** a **max**, které budou obsahovat nejnižší a nejvyšší pracovitost z mediánů **mid_val (pro sudý počet N je MAX největší z prvků na indexu N/2)** všech krajů. Bude třeba také držet **range_count** - celkový počet všech poddaných napříč rozsahy pro všechny kraje. Jednoduše tak sečteme všechny hodnoty **left_mid_count** spolu s **mid_right_count** + 3 pro každý kraj (naši m_mediáni kraje). Přičítat konstantní počet budeme pouze v prvních K průchodech, jelikož jakmile budeme následně rozsahy zmenšovat, nastanou situace, kdy se mezi hranice **min** a **max** nemusí vejít ani počet všech známých **m_mediánů** napříč kraji.

Tady najednou přejdeš k proměnným potřebným k binárnímu vyhledávání a k proměnné range_count, kterou bys snadno mohl spočítat až během určování odhadu.

Jakmile máme rozsahy získané, můžeme konečně udělat náš první odhad.

Teď přejdeš k určování odhadu.

Hned první postup, získání odhadu, který mě napadl, bylo jednoduché sečtení **min** a **max** a následné vydělení 2. Nabízí se také možnost najít mezi celým královstvím nejmenší a největší prvek, nebo čistě sečíst hodnoty 1 a **Pmax** a následně dělit 2, najít počet menších a větších prvků než je náš odhad a buďto se vydat do pravé nebo levé strany podle toho, která je při našem odhadu větší a proces opakovat.

Rychle popsat jednodušší a pomalejší algoritmus může být dobrý nápad, pak dostaneš alespoň nějaké body, pokud tvůj složitější algoritmus nefunguje. Varianta s průměrováním min a max, kde na začátku min = 1 a max = Pmax, je vzorové řešení. Hodilo by se ale alespoň určit složitost.

Tento přístup by fungoval a vzhledem k neustálému půlení intervalu (nebo alespoň jejich hodnot) by nakonec medián našel. To bych ale nebyl já, kdyby mě netrápila jedna zásadní věc. Mohou nastat situace, kde, ať už při výběru **min** a **max**, jakožto nejmenší a největší z

7281312229027 Radek Zach 38-1-3 2/11

mediánů každého kraje, nebo pouhé využití minimálních a maximálních hodnot (1, **Pmax**), bude neefektivní pokud tyto krajní hodnoty budou mít udržovaném intervalu kvůli malému výskytu zanedbatelnou skutečnou váhu, avšak budou dosahovat oproti zbytku “extrémních” hodnot. A to i pro **min** a **max** z našich mediánů. Pokud například budeme mít v tomto rozsahu koncový úsek, který bude obsahovat v porovnání se zbytkem extrémní hodnoty (jednotlivé hodnoty K, N a Pmax se liší řadově) a rozhodneme se využít hledání středu binární metodou, bude náš výsledek těmito extrémy velice ovlivněn. S tím jsem nebyl spokojený a po chvilce logické úvahy bylo jasné, že bude třeba jiného přístupu.

Formálně bude složitost $\mathcal{O}$(K log Pmax), protože každá iterace zmenší rozdíl mezi min a max alespoň na polovinu.

Vezměme si následují příklad, představte si, že jste získali rozsah, u kterého hodnoty v něm obsažené vypadají nejak takto {1,3,5,8,11,12,59409}. Pokud bychom chtěli odhad vypočítat pouhým sečtením krajních hodnot a získáním poloviny, vyšlo by nám 29702~~,~~5. V takovém případě bychom se po prvním kroku nedostali vůbec nikam. Pokud bychom při hledání počtu menších a větších prvků si alespoň pamatovali k odhadu krajní hodnoty, dokázali bychom po výsledném porovnání počtu větších, menších prvků interval maximálně snížit o 1 pozici v řadě. O pouhém přičtení, odečtení 1 ani nemluvě.

Vysvětlit, proč je jiný algoritmus pomalý, není v řešení potřeba, ale ničemu to nevadí. Navíc je vhodné si to rozmyslet, pokud chceš najít rychlejší algoritmus. Nicméně dva odstavce s příkladem jsou asi opravdu zbytečně dlouhé.

Jak to ale efektivně vyřešit i v případě limitu délky zprávy, která musí být konstantní? Uvědomíme si, že i pro konstantní počet hodnot můžeme udělat mnohem efektivnější odhad zahrnující váhu respektive hustotu našich rozsahů hodnot pro hledaný medián. Bohužel, jelikož v začátku pouze načítáme hodnoty tak přesnou pozici mediánu nemůžeme určit. Proto pozici mediánu odhadneme získáním pozice středu rozsahu z všech krajů.

Nejsem si jistý, co se tímto odstavcem snažíš říci.

Plný potenciál můžeme využít až v případě, kdy známe celkovou délku do našeho **min**. Této větě nerozumím. Samotnou délku (počet pozic) můžeme zjistit jednoduše a to tak, že po prvních K průchodech, kdy už budeme vědět min. a max, se pustíme i s naším prvním odhadem mediánu do dalších K průchodů s cílem zjistit počet prvků menších, větších než náš odhad. Mezitím si také budeme pamatovat počet všech prvků, které jsou menší než min. Označme si tento počet **count_to_min** (tvorba názvů v angličtině mi nikdy nešla, za to se omlouvám) a udržujme si součet napříč kraji.

Možná jej jen špatně chápu, ale zdá se mi, že tento odstavec neříká nic jiného než „Spočítáme počet prvků menších než min.“

V neposlední řadě bude také třeba aktualizovat naše rozsahy novými hodnotami. A jelikož víme, že se medián nachází v intervalu **min** a **max**, budeme si pro něj udržovat jeho velikost. Já jsem zvolil pro levou hranici rozsahu **left_val** ← **P[i]** v kraji, které je první větší nebo rovno **min**. Pravá hranice **right_val** ← **P[i]** bude hodnota menší nebo rovna **max**. Pro **mid_val** si budu v kraji uchovávat takové **P[i]**, jež je středem krajních hodnot. Pro zbylé dvě proměnné v **Range** získáme opět počty poddaných mezi těmito pracovitostmi. Hodnoty **range_count** budou obsahovat hodnoty stejným způsobem, jako v prvních K průchodech (sumu počtu mezi rozsahy). Další tři proměnné, které přibudou **e_smaller**, **e_larger** a **correct_median**.

Počkat, takže v první iteraci představovaly hodnoty left_val, mid_val a right_val po řadě 25., 50. a 75. percentil, ale v dalších budou představovat 0., 50. a 100. percentil (ze zbývajících hodnot)? Proč? Obecně se mi zdá, že první iteraci považuješ mnohde za speciální, a nevidím k tomu důvod.

E_smaller bude uchovávat hodnotu pracovitosti napříč všemi K, která je největší nižší než odhad mediánu, **e_larger** naopak nejnižší vyšší a **correct_median** pracovitost v případě, že si je s odhadem rovna. Velikost zprávy po prvních K průchodech tak bude 12 konstantních celočíselných proměnných.

Teď se od počítání odhadu vracíš k implementaci binárního vyhledávání.

V momentě, kdy budem znát počty poddaných s P menším a větším než odhad, zjistíme jejich absolutní hodnotu rozdílu. Pak díky tomu, že si udržujeme pracovitosti s hodnotou

7281312229034 Radek Zach 38-1-3 3/11

nejvyšší nižší a nejnižší vyšší, může se jejich počet pro sudé lišit až o 3 poddané a pro liché o 2 poddané → podle toho která strana převažuje, vrátíme jako výsledek pracovitost menší nebo nižší, respektive tu na převažující straně, pokud je K*N liché a počty jsou si rovny vrátíme **correct_median**, v případě rovnosti u sudého počtu **e_smaller** (můžeme i **e_larger**)).

Pokud se počty liší rozdílem více než je přípustné, vypočítáme opět náš nový odhad mediánu. Teď už ale způsobem ve kterém zohledníme celkový počet poddaných < **min** spolu s faktem, že pozice medianu je pro sudá K*N/2, lichá K*N/2+1. Proto jednoduše počet **count_to_min** odečteme od polovičního počtu všech poddaných a získáme vzdálenost v počtech poddaných od **min** k našemu mediánu. Abychom zajistili, že program skončí a dá správný výsledek, budeme vždy po K průchodech upravovat **min** nebo **max** podle toho, která strana byla početně vyšší. V případě většího počtu podaných s nižší pracovitostí aktualizujeme hodnotu **max** ← **e_smaller**, opačně pak **min** ← **e_larger**.

Předáme tak získané rozsahy, vzdálenost mediánu od **min** a celkový počet prvků v rozsahu, funkci.

Teď už tedy ke způsobu získání odhadu mediánu.

Tohle je podle mě nejdůležitější část algoritmu a osobně bych s ní začal, po krátkém odstavci o binárním vyhledávání a o tom, jak se rozhodnout do které půlky se vydat. Jak získat z krajů potřebné hodnoty bych dal až na konec, a moc bych to nerozebíral. Nicméně mám jako organizátor výhodu v tom, že vím, v jaké části se jednotlivá řešení liší…

Jelikož se rozsahy mohou překrývat a extrémní hodnoty mohou vést k nepřesnostem pro aritmetický průměr nebo středové hodnoty, chceme první skutečnost využít, a druhou co nejvíce tou první eliminovat. A právě na to se nejvíce hodí kumulativní distribuční funkce.

Nikde ve svém řešení nepopisuješ, co máš kumulativní distribuční funkcí na mysli. CDF je sice obecně známý termín, ale tady ho používáš v úplně jiném kontextu – obecně CDF vždy patří k nějaké náhodné veličině.
Tady se zdá se jedná o CDF pracovitosti náhodně vybraného poddaného, ale jen o její aproximaci, kde je každý kraj odhadnut jako dvojice intervalů s uniformní distribucí. Navíc v první iteraci uvažujeme jen poddané, které jsou v prostředních dvou čtvrtinách svého kraje?

Extrémní výhodou využití této funkce pro náš problém je, že řeší všechny problémy neúměrného váhového zkreslení dat, které jsem popsal výše. **Proč?** Funkce prochází po bodech (poddaných ze všech K rozsahů setříděných vzestupně) z leva do prava a v případě, že v bodě se nenachází žádný aktivní rozsah, pouze přičte skokovou konstantu, nazvěme ji **step** a reprezentuje podíl jednoho poddaného v celém rozsahu (**min** a **max**) napříč kraji → **step = 1 / range_count.**

V situacích kdy dojde k překrývání rozsahů poddaných napříč kraji, tzv. hustota se přičítá. Ta zjednodušeně reprezentuje, **kolik poddaných v intervalu, u kterých nevíme přesnou pracovitost, se nachází mezi body (poddanými left_val, mid_val, right_val)**

Stručně: Projdeme všech K rozsahů, zjistíme zda hodnoty K[i]: left_mid_count a mid_right_count > 0. Pokud ne, rozsah pro určitý kraj zde nemůže existovat → načteme pouze nenulové body. Pokud se poddaní mezi nachází, vytvoříme mezi jejich krajními poddanými interval.

K tomu abychom poddané mohli procházet a určit, které z rozsahů pro náš bod v aktuální chvíli platí inicializujeme si dva kontejnery (unordered_map) **start** a **end**, které umožní instantní přístup podle klíče P[i]. Jeden bude představovat začátek intervalu a druhý jeho konec a budou obsahovat společnou hustotu kterou rozsah nabývá (interval je zleva

7281312229041 Radek Zach 38-1-3 4/11

uzavřený, zprava otevřený a vždy tvořen pouze poddanými stejného kraje **<left_val, mid_val)**, **<mid_val, right_val))**. Pokud jsou podmínky pro rozsah splněny, vypočítáme jejich hustotu tak, že počet prvků v intervalu vydělíme celkovým počtem všech prvků čímž získáme váhové zastoupení. Poté vydělíme rozdílem krajních prvků, čímž získáme úhel sklonu (míru hustoty), který bude určovat rychlost nárůstu hodnoty.

Následně nám stačí už jen body vzestupně seřadit a můžeme začít procházet. Bod, který budeme chtít dosáhnout **med_pos** kumulováním pravděpodobnosti na ni převedeme tak, že vydělíme jeho pozici celkovým počtem všech poddaných (**range_count**).

Postupným procházením bodů od levé části budeme kumulovat hodnotu **c_sum** až do hodnoty **med_pos**. Jakmile ji dosáhneme zkontrolujeme, zda je některý z intervalů aktivní. Pokud není, znamená to, že kvantil se nachází v přímo v bodě, kde se nachází náš poddaný s pracovitostí P[i] → náš nový odhad mediánu. Vždy když projdeme bodem, zkontrolujeme zda se v **end.at(P[i])** nenachází konec intervalu. Pokud nachází, prvně odečteme jeho hustotu ze součtu aktivních intervalů (interval je zprava otevřený!!), říkejme mu **active_sum**. Následně zjistíme, zda v tom samém bodě nezačíná nový. Pokud ano, naopak jej přičteme.

Zdá se mi, že se tyhle podmínky překrývají.

Každým průchodem bodu spočítáme celkovou míru skoku **jump** mezi body (i celým intervalem), tedy P[i+1] - P[i] a poté vynásobíme všemi aktivními intervaly.

V tento moment zkontrolujeme, zda nebyla hodnota **med_pos** dosažena **(jump + c_sum)**, pokud ano, znamená to, že se hodnota odhadu mediánu nachází v aktuálním intervalu (**jump** je nárůst o celý bod tedy i interval). Abychom zjistili přesnou pozici v tomto intervalu, respektive odhadli hodnotu mezi P[i] a P[i+1], jednoduše získáme rozdíl hledaného bodu odečtením **c_sum** od **med_pos** (ještě před přičtením **jump**), který poté vydělíme **active_sum**, pro získání změny nárůstu a přičteme **P[i]** jako základ. Konečně pak odhad mediánu funkce vrátí.

Pokud hledaný bod dosažen nebyl přičteme k součtu **c_sum** skok o celý interval a zjistíme, zda nárůst o **step** nedosáhne hledané hodnoty. Pokud ano, vrátíme příští bod P[i+1]. Jinak k součtu přičteme **step** a pokračujeme v další iteraci, dokud **med_pos** nedosáhneme nebo neprojdeme všechny body. Pokud tato situace nastane, vrátí se 0.

Důkaz správnosti a odhad asymptotické složitosti počtu vyslání posla:

Algoritmus najde správný medián vždy. Při prvních K průchodech získá ze všech krajů jejich medián, dolní a horní čtvrtinu. Hodnoty **min** a **max** následně budou obsahovat nejnižší a nejvyšší z mediánů. Navíc pro (N%2 je rovno 0) bude **max** obsahovat největší z hodnot na pozici N/2. Tím skutečně zajistím, že pravý medián $\in [min, max]$, protože jak pro maximum, tak i pro minimum platí, že alespoň polovina poddaných kraje bude N/2 < min a zároveň max < N/2. Tím zaručím, že se v intervalu medián nachází.

Mohl jsi prostě nastavit na začátku min=1 a max=Pmax a tuhle část neřešit, tvá volba min a max žádné výrazné zrychlení nepřináší.

Hodnota mediánu může být odhadnuta pouze z hodnot, které kromě prvních K průchodů nepřesahují **min** a **max**. Pokud by odhad mediánu po prvních K průchodech byl mimo rozsah, ničemu by to nevadilo, jelikož ve stejnou chvíli, kdy k vyhodnocování úspěšnosti

7281312229058 Radek Zach 38-1-3 5/11

“špatného” odhadu dochází, hodnoty min a max nemohou být do konce dalších K průchodu mediánem ovlivněny. Díky tomu, tak získáváme rozsahy podle těchto hodnot. Jakmile se min a max přizpůsobí nepřesným výsledkům tj. strana s vyšším počtem poddaných přesune hranici pro rozsah stejné strany právě o hodnotu poddaného jehož pracovitost je blíže k středu (směrem dovnitř), což pouze zapříčiní korekci mediánu nacházejícího se pak v rozsahu (mezi min a max), každopádně nový odhad mediánu už bude získán na základě předchozích hodnot, které splňovaly podmínku nejmenší a největší z mediánů každého kraje. Díky tomu už nebude možné odhad mediánu dostat mimo rozsah, jelikož v případě, kdy funkce nespočítá medián správně, nejhorší situace, co může nastat, je, že funkce jako odhad vrátí nulu. To nastane ale jen v situaci, kdy počet bodů mezi min a max je 0. A tato situace nikdy nemůže nastat, jelikož se rozsah posouvá jen přes poddané, kteří mají pracovitost o jednu pozici menší nebo větší než odhad (min a max vždy obsahují nějakou skutečnou hodnotu P).

Této části nerozumím.

Tudíž podle definice naší vážené spojitéenkumulativní distribuční funkce nikdy nemůže její návratová hodnota přesáhnout max, nebo min pokud není velikost rozsahu 0. To společně s posouváním hranic min a max alespoň o jednu pozici v řadě vždy směrem kratší strana → delší strana, kdykoliv jsou počty určující pozici odhadu mediánu vyšší jak 3 pro sudé N, 2 pro liché, musíme se v určitý moment rozsahy přiblížit natolik, abychom medián vrátili.

Bylo by vhodné alespoň stručně zdůvodnit, že medián v [min, max] zůstane.

Asymptotická složitost počtu vyslání posla bude v nejhorším případě $O(K \cdot log_2 N)$. Jelikož řešení nezkracuje délku intervalu půlením Pmax, ale vždy získá pozici mediánu, pro kterou počítá pravděpodobnost výskytu v bodě. Nebude to tedy Pmax, co bude zapříčiňovat nepřesnost odhadu, ale právě délka intervalu poddaných mezi krajními body.

To ale ještě vůbec neznamená, že je složitost $\mathcal{O}$(K log N). Kdyby tvůj algoritmus například pracovitosti jen porovnával, tak by se z toho dalo odvodit, že složitost nemůže záviset na Pmax. Nicméně tvůj algoritmus s pracovitostmi dělá i aritmetické operace. Hlavně to, že složitost nezávisí na Pmax, jetě neznamená, že bude $\mathcal{O}$(K log N).

Pokud se budeme bavit o časové složitosti algoritmu, tak naprogramované řešení které přiložím poběží v nejhorším případě O(KN). To je z důvodu, protože jsem v archivech iteroval přes všechny poddané. Samozřejmě toto by se dalo zlepšit jednoduchým binárním vyhledáváním. Ale cílem spíše bylo vědět, že výpočet odhadu pravděpodobnosti výskytu funguje. Řešení jsem zkoušel projet i 100 000 náhodně generovanými vstupy pro K = 15, N = 100, Pmax = 20000. A pro K = 800 cca 10000x. Nejhorší počet vyslání byl 7K. Toto řešení bude mít asymptotickou složitost podobnou jako ostatní lepší řešení, avšak průměrná bude extrémně rychlá. I docela často jsem pro K = 100 byl schopen medián najít během 2K, 3K. Navíc, byly bychom schopni přesnost ještě vylepšit jednoduše zvětšením statistického vzorku.

$\mathcal{O}$(KN) by bylo potřeba jen na načtení vstupu, nebo na podívání se na všechny poddané. Pochybuji, že by šlo najít medián rychleji. Navíc tvůj algoritmus v $\mathcal{O}$(KN) neběží, bude to spíše $\mathcal{O}$(N) krát počet vyslání poslů. Průměrná časová složitost dává smysl jen pro algoritmy, které používají náhodu, což tvůj nedělá. (Až na hashovací tabulky, ale ty hádám nemáš na mysli.) Můžeš počítat průměr přes všechny vstupy, ale to se typicky nedělá, protože tvé vstupy asi nebudou dokonale náhodně vygenerované.

Ještě bych chtěl dodat, že to je poprvé, co takto teoretické úlohy odevzdávám. Proto bych byl rád i za zpětnou vazbu, co se popisu řešení týče. Uvědomuji si, že je to dlouhé a některé části možná až moc zbytečně vysvětluji i kostrbatě popisuji. Jen chci mít jistotu, že jsem popsal vše.

Nějakou zpětnou vazbu k řešení jsem dopsal. Nechci o sobě tvrdit, že jsem autorita na stylistiku řešení, takže vše ber s rezervou. Obecně doporučuji se snažit vysvětlit nejen to, co děláš, ale také čeho se tím snažíš docílit. Dokazovat, že algoritmus funguje, je často nejjednodušší po částech. Korektnost binárního vyhledávání třeba nezáleží na tom, jaký zvolíme uvnitř intervalu prvek, takže můžeš jeho volbu analyzovat zcela odděleně.

Níže vkládám kód v cpp. Kdyžtak se omlouvám za vzhled, lépe to vložit nešlo.

Vůbec nedokazuješ, že tvůj algoritmus má takovou složitost, jakou uvádíš. Je potřeba opravdu dokázat, že pro každý vstup doběhne dostatečně rychle, nestačí, že vyřeší všechny tvary vstupů, které tebe napadly. Navíc neplatí, že by tvůj algoritmus v každém kroku zahodil polovinu všech prvků, nebo jiný konstantní zlomek. Může se stát, že v jednom kroku zahodí jen K + N poddaných, pokud bude vstup vypadat zhruba takto:

Kraj 1:	1	100	101	102	...
Kraj 2:	2	1000000	1000001	1000003	...
Kraj 3:	3	1000100	1000101	1000103	...
Kraj 4:	4	1000200	1000201	1000203	...
Kraj 5:	5	1000300	1000301	1000303	...
...					

Odhad mediánu pomocí CDF ti pak vyjde těsně pod 1000000, protože levý interval 2. kraje a dál povede od ~1 po ~1000000, a díky 1. kraji dosáhneme hodnoty 0,5 ještě před koncem těchto levých intervalů.

7281312229065 Radek Zach 38-1-3 6/11

```
struct Range {long left_val = 0; long middle_val; long left_mid_count; long mid_right_count;
long right_val;};

long double medianEstimateUsingWECDF(vector<Range> ranges, double steps_to_median, double
range_size) {
    const long double single_step = 1 / range_size;
    long double median_probability_pos = steps_to_median / range_size;
    long double weights_sum = 0;

    unordered_map<long, long double> start_density_sum;
    unordered_map<long, long double> end_density_sum;
    vector<long> points;

    for (int i = 0; i < ranges.size(); ++i) {

        if (ranges[i].left_val == 0) continue;
        long smallest_point = ranges[i].left_val;
        long middle_point = ranges[i].middle_val;
        long largest_point = ranges[i].right_val;
        long numbers_count = 1;

        points.push_back(smallest_point);
        if (middle_point != 0) {
            points.push_back(middle_point);
            numbers_count += 1;
        }
        if (largest_point != 0) {
            points.push_back(largest_point);
            numbers_count += 1;
        }

        if (middle_point != 0 && ranges[i].left_mid_count > 0 && middle_point > smallest_point)
{
            long double density = ranges[i].left_mid_count / range_size / (middle_point -
smallest_point);
            start_density_sum[smallest_point] += density;
            end_density_sum[middle_point] += density;
            numbers_count += ranges[i].left_mid_count;
        }
        if (middle_point != 0 && largest_point != 0 && ranges[i].mid_right_count > 0 &&
largest_point > middle_point) {
            long double density = ranges[i].mid_right_count / range_size / (largest_point -
middle_point);
            start_density_sum[middle_point] += density;
            end_density_sum[largest_point] += density;
            numbers_count += ranges[i].mid_right_count;
```

7281312229072 Radek Zach 38-1-3 7/11

```
        }
        long double weight = numbers_count / range_size;
        weights_sum += weight;
    }
    sort(points.begin(), points.end());
    if (points.empty()) {
        cout << "Velikost rozsahů je 0!" << endl;
        return 0;
    }

    const long double round_diff = 1e-18L;
    if (median_probability_pos <= round_diff) return (long double) points.front();
    if (median_probability_pos >= 1.0L - round_diff) return (long double) points.back();

    long double norm_value = 1 / weights_sum;
    for (auto& it : start_density_sum) it.second *= norm_value;
    for (auto& it : end_density_sum) it.second *= norm_value;

    long double active_interval_densities_sum = 0;
    long double cumulative_sum = 0;

        for (int i = 0; i < points.size() - 1; ++i) {
            if (end_density_sum.find(points[i]) != end_density_sum.end())
active_interval_densities_sum -= end_density_sum[points[i]];
            if (start_density_sum.find(points[i]) != start_density_sum.end())
active_interval_densities_sum += start_density_sum[points[i]];
            long double value_incerease = (long double)(points[i + 1] - points[i]) *
active_interval_densities_sum;

            if (value_incerease + cumulative_sum >= median_probability_pos &&
active_interval_densities_sum > 0) {
                long double diference = median_probability_pos - cumulative_sum;
                long double median =  diference / active_interval_densities_sum + points[i];
                return median;
            }
            cumulative_sum += value_incerease;

            if (cumulative_sum < median_probability_pos && cumulative_sum + single_step >=
median_probability_pos) return (long double)points[i + 1];
            cumulative_sum += single_step;
        }


        return points.back();

}
```

7281312229089 Radek Zach 38-1-3 8/11

```
pair<long, long> medianInTheKingdom(vector<vector<long>> input, const int K, const int N) {
    long expeditionsCount = 0, smallest = LONG_MAX, largest = LONG_MIN, median_guess = 0;
    bool dataLoaded = false, countsLoaded = false;
    while (true) {

        long between_medians_count = 0, from_start_to_smallest = 0;
        long count_of_smaller = 0, count_of_larger = 0;
        long existing_smaller = 1, existing_larger = LONG_MAX, correct_median = 0;

        vector<Range> ranges(K);
        for (int i = 0; i < K; ++i) {
            vector<long> values;
            if (!dataLoaded) {
                int left = N / 4, right = 3 * N / 4, mid = (N%2 == 0) ? N / 2-1 : N / 2, mid2 =
N / 2;
                ranges[i].left_val = input[i][left];
                ranges[i].middle_val = input[i][mid];
                ranges[i].right_val = input[i][right];
                ranges[i].left_mid_count = max(0, mid - left - 1);
                ranges[i].mid_right_count = max(0, right - mid - 1);
                between_medians_count += ranges[i].left_mid_count + ranges[i].mid_right_count +
3;

                if (smallest > input[i][mid]) smallest = input[i][mid];

                if (largest < input[i][mid2] && N%2 == 0) {
                    largest = input[i][mid2];
                } else if (largest < input[i][mid] && N%2 != 0) {
                    largest = input[i][mid];
                }
                if (i == K - 1) {
                    median_guess = medianEstimateUsingWECDF(ranges,(long
double)(between_medians_count)/2 ,(long double)between_medians_count);
                    dataLoaded = true;
                }
                expeditionsCount++;

            }
            else {
                    for (int j = 0; j < N; ++j) {
                        if (input[i][j] < median_guess) {
                            count_of_smaller++;
                            if (existing_smaller < input[i][j]) existing_smaller = input[i][j]
                        }
```

7281312229096 Radek Zach 38-1-3 9/11

```
                        else if (input[i][j] > median_guess) {
                            count_of_larger++;
                            if (existing_larger > input[i][j]) existing_larger = input[i][j];
                        }
                        else correct_median = input[i][j];

                        if (input[i][j] >= smallest && input[i][j] <= largest) {
Tohle se nikdy nestane, ne?  if (ranges[i].left_val == 0) ranges[i].left_val = input[i][j];
                            else {
                                ranges[i].right_val = input[i][j];
                                values.push_back(input[i][j]);
                            }
                            between_medians_count++;
                        }
                        else if (input[i][j] < smallest) from_start_to_smallest++;


                        if (j == N-1) {
                            if (values.size() > 1) {
                                if (values.size() <= 3) {
                                    values.pop_back();
                                    ranges[i].middle_val = values[0];
                                    ranges[i].left_mid_count = 0;
                                    ranges[i].mid_right_count = (values.size() == 2) ? 1 : 0;
                                }
                                else {
                                    values.pop_back();
                                    ranges[i].middle_val = (values.size()%2 == 0) ?
values[values.size()/2-1] : values[values.size() / 2];
                                    ranges[i].left_mid_count = (values.size()%2 == 0) ?
values.size() / 2 - 1 : values.size() / 2;
                                    ranges[i].mid_right_count = values.size() / 2;
                                }
                            }
                            if (i == K-1) {
                                countsLoaded = true;
                            }
                        }
                    }

                expeditionsCount++;
            }
        }
        if (countsLoaded) {
            if ((K*N % 2 == 0 && abs(count_of_smaller - count_of_larger) > 3) ||
                (K*N % 2 != 0 && abs(count_of_smaller - count_of_larger) > 2)) {
```

7281312229102 Radek Zach 38-1-3 10/11

```
                long middle = (K*N % 2 == 0) ? (K*N / 2) : (K*N / 2 + 1);
                long steps_forward_needed = middle - from_start_to_smallest;
                double median_estimate = medianEstimateUsingWECDF(ranges, steps_forward_needed,
between_medians_count);
                if (median_estimate == 0) {
                    if (count_of_smaller > count_of_larger) {
                        median_guess = existing_smaller;
                    }
                    else if (count_of_larger > count_of_smaller) {
                        median_guess = existing_larger;
                    }
                }
                else {
                    if (existing_larger != INT_MAX && count_of_smaller < count_of_larger)
smallest = existing_larger;
                    else if (existing_smaller != 0 && count_of_smaller > count_of_larger)
largest = existing_smaller;
                    median_guess = static_cast<long>(median_estimate + 0.5);
                }

            }
            else {

                if (count_of_smaller > count_of_larger || (count_of_smaller == count_of_larger
&& K*N%2 == 0)) return make_pair(existing_smaller, expeditionsCount);
                if (count_of_smaller < count_of_larger) return make_pair(existing_larger,
expeditionsCount);
                return make_pair(correct_median, expeditionsCount);
            }
        }
    }
}
```